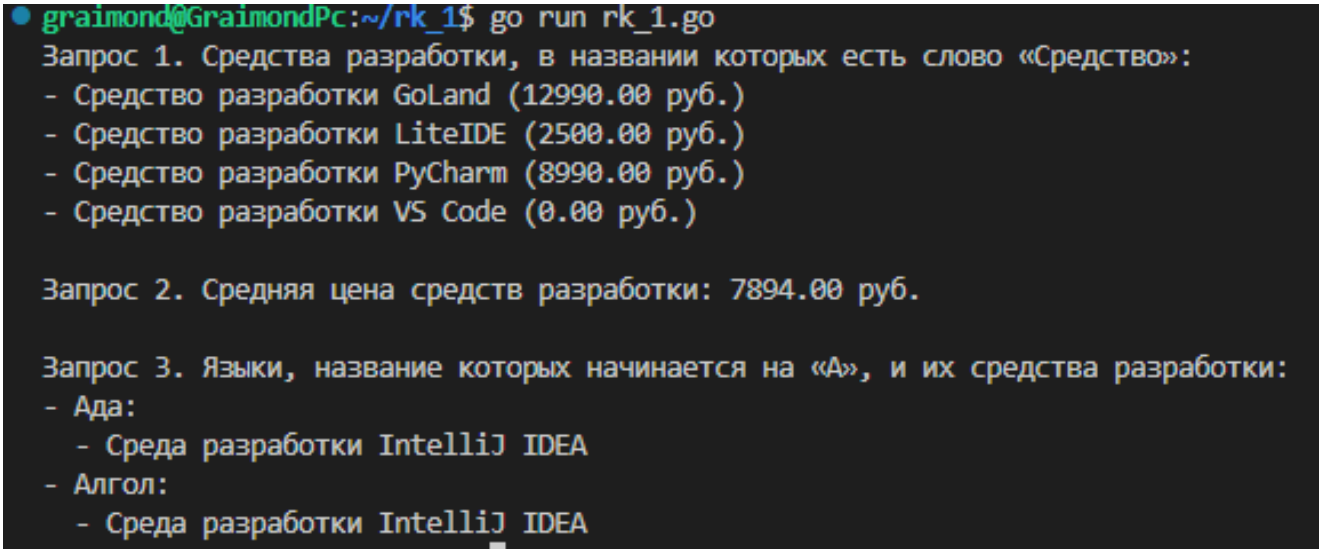
Результат программы



Программа rk_1.go

```
package main

import (
 "fmt"
 "sort"
 "strings"
)

type ProgrammingLanguage struct {
 ID   int
 Name string
}

type DevelopmentTool struct {
 ID    int
 Name  string
 Price float64
}

type LanguageTool struct {
 LanguageID int
 ToolID     int
}

func main() {
 languages := []ProgrammingLanguage{
  {ID: 1, Name: "Go"},
  {ID: 2, Name: "Python"},
  {ID: 3, Name: "Алгол"},
  {ID: 4, Name: "Ада"},
  {ID: 5, Name: "Java"},
 }

 tools := []DevelopmentTool{
  {ID: 1, Name: "Средство разработки GoLand", Price: 12990},
  {ID: 2, Name: "Средство разработки PyCharm", Price: 8990},
  {ID: 3, Name: "Среда разработки IntelliJ IDEA", Price: 14990},
  {ID: 4, Name: "Средство разработки VS Code", Price: 0},
  {ID: 5, Name: "Средство разработки LiteIDE", Price: 2500},
 }

 links := []LanguageTool{
  {LanguageID: 1, ToolID: 1},
  {LanguageID: 1, ToolID: 3},
  {LanguageID: 1, ToolID: 4},
  {LanguageID: 1, ToolID: 5},
  {LanguageID: 2, ToolID: 2},
  {LanguageID: 2, ToolID: 3},
  {LanguageID: 2, ToolID: 4},
  {LanguageID: 3, ToolID: 3},
  {LanguageID: 4, ToolID: 3},
  {LanguageID: 5, ToolID: 3},
 }

 languagesByID := make(map[int]ProgrammingLanguage, len(languages))
```

```
 for _, language := range languages {
  languagesByID[language.ID] = language
 }
 toolsByID := make(map[int]DevelopmentTool, len(tools))
 for _, tool := range tools {
  toolsByID[tool.ID] = tool
 }

 toolsByLanguage := make(map[int][]DevelopmentTool)
 languagesByTool := make(map[int][]ProgrammingLanguage)
 for _, link := range links {
  language, languageOK := languagesByID[link.LanguageID]
  tool, toolOK := toolsByID[link.ToolID]
  if !languageOK || !toolOK {
   continue
  }
  toolsByLanguage[language.ID] = append(toolsByLanguage[language.ID], tool)
  languagesByTool[tool.ID] = append(languagesByTool[tool.ID], language)
 }

 var matchingTools []DevelopmentTool
 for _, tool := range tools {
  if strings.Contains(strings.ToLower(tool.Name), "средство") {
   matchingTools = append(matchingTools, tool)
  }
 }
 sort.Slice(matchingTools, func(i, j int) bool {
  return matchingTools[i].Name < matchingTools[j].Name
 })
 fmt.Println("Запрос 1. Средства разработки, в названии которых есть слово «Средство»:")
 for _, tool := range matchingTools {
  fmt.Printf("- %s (%.2f руб.)\n", tool.Name, tool.Price)
 }

 var totalPrice float64
 for _, tool := range tools {
  totalPrice += tool.Price
 }
 if len(tools) > 0 {
  averagePrice := totalPrice / float64(len(tools))
  fmt.Printf("\nЗапрос 2. Средняя цена средств разработки: %.2f руб.\n", averagePrice)
 } else {
  fmt.Println("\nЗапрос 2. Средняя цена не вычисляется: список средств разработки пуст.")
 }

 var matchingLanguages []ProgrammingLanguage
 for _, language := range languages {
  if strings.HasPrefix(strings.ToLower(language.Name), "a") {
   matchingLanguages = append(matchingLanguages, language)
  }
 }
 sort.Slice(matchingLanguages, func(i, j int) bool {
  return matchingLanguages[i].Name < matchingLanguages[j].Name
 })
 fmt.Println("\nЗапрос 3. Языки, название которых начинается на «A», и их средства разработки:")
 for _, language := range matchingLanguages {
  fmt.Printf("- %s:\n", language.Name)
  associatedTools := toolsByLanguage[language.ID]
  if len(associatedTools) == 0 {
   fmt.Println("  (нет связанных средств разработки)")
   continue
  }
  sort.Slice(associatedTools, func(i, j int) bool {
   return associatedTools[i].Name < associatedTools[j].Name
  })
  for _, tool := range associatedTools {
   fmt.Printf("  - %s\n", tool.Name)
  }
 }

 _ = languagesByTool
}
```